06 Dsa

Binary Search

When to Use

Keywords: `sorted array`, `sorted`, `threshold`, `first occurrence`, `last occurrence`, `find position`, `O(log n)`, `monotonic`

Input is sorted (or monotonic), and you need to find a position, threshold, or boundary.

Variants

Variant	Description
Standard	Find exact match in sorted array
Lower bound	First position where value >= target
Upper bound	First position where value > target
Binary search on answer	Search over possible answers, check feasibility

Classic Problems

- Binary search in sorted array
- First and last position of element
- Search insert position
- Capacity to ship packages within D days (binary search on answer)
- Koko eating bananas

Template — Standard

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1

    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1

    return -1  # not found
```

Template — First Occurrence (Lower Bound)

```
def first_occurrence(arr, target):
    lo, hi = 0, len(arr) - 1
    result = -1

    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            result = mid
            hi = mid - 1  # keep searching left
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1

    return result
```

Template — Upper Bound (First element > target)

```
def upper_bound(arr, target):
    lo, hi = 0, len(arr) - 1
    result = -1

    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] > target:
            result = mid
            hi = mid - 1
        else:
            lo = mid + 1

    return result
```

Complexity

Time: O(log N)
Space: O(1)

DFS / BFS (Trees & Graphs)

When to Use

Keywords: `tree`, `hierarchy`, `traverse`, `path`, `connected`, `level`, `depth`, `adjacency`, `graph`

Pattern	Use When
DFS	Explore deep paths, backtracking, topological sort, tree traversal
BFS	Shortest path (unweighted), level-order traversal, nearest neighbors

DFS Variants

Variant	Description
Pre-order	Root → Left → Right
In-order	Left → Root → Right (gives sorted order in BST)
Post-order	Left → Right → Root (useful for deletion, subtree computation)

Classic Problems

- Tree traversals (inorder, preorder, postorder)
- Maximum depth of tree
- Level order traversal (BFS)
- Number of islands (DFS/BFS on grid)
- Word ladder (BFS shortest path)
- Course schedule (topological sort)

Template — DFS on Tree

```
def dfs(node):
    if not node:
        return

    # process node (pre-order)
    dfs(node.left)
    # process node (in-order)
    dfs(node.right)
    # process node (post-order)
```

Template — BFS Level Order

```
from collections import deque

def bfs_level_order(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level)

    return result
```

Complexity

Time: O(V + E) where V = vertices, E = edges
Space: O(V) for visited set / queue / recursion stack

DFS on Trees: The Mental Model

The Biggest Mistake Beginners Make

When solving tree problems, most people think:

> "I need to traverse the tree."

That's not quite right. A better way to think is:

> "Every node asks its children for information."

The entire DFS pattern is built around this idea.

Tree Representation

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
```

Example tree:

```
      1
     / \
    2   3
   / \
  4   5
```

`node.left` points to another `TreeNode`. `node.right` points to another `TreeNode`. The picture is simply a visualization of these pointers.

What is a Subtree?

A subtree is a node and everything below it.

```
Whole tree:          Subtree at 2:       Subtree at 4:
      1                    2                   4
     / \                  / \
    2   3                4   5
   / \
  4   5
```

The DFS Recipe

Every DFS problem follows the same structure.

Step 1: Decide what information you want each child to return

Problem	Child returns
Max Depth	Depth of its subtree
Count Nodes	Number of nodes in its subtree
Tree Sum	Sum of all values in its subtree
Maximum Value	Maximum value inside its subtree
Search	True / False

Step 2: Base Case

Ask: "What should an empty tree return?"

Problem	`None` returns
Max Depth	0
Count Nodes	0
Tree Sum	0
Search	False

Step 3: Ask Both Children

```
left_answer = solve(left subtree)
right_answer = solve(right subtree)
```

Not actual values — the COMPLETE answer for that subtree.

Step 4: Combine

This is the only part that changes from problem to problem.

Worked Examples

Maximum Depth

```
Node 4 (leaf): returns 1  (1 + max(0, 0))
Node 2: left_depth=1, right_depth=1 → returns 1 + max(1, 1) = 2
```

```
return 1 + max(left_depth, right_depth)
```

Count Nodes

```
Node 2: left_count=1, right_count=1 → returns 1 + 1 + 1 = 3
```

```
return 1 + left_count + right_count
```

Tree Sum

```
Node 2: left_sum=4, right_sum=5 → returns 2 + 4 + 5 = 11
```

```
return node.val + left_sum + right_sum
```

Maximum Value

```
Node 5: left_max=100, right_max=3 → returns max(5, 100, 3) = 100
```

```
return max(node.val, left_max, right_max)
```

Search

```
Children return True/False
```

```
return node.val == target or found_left or found_right
```

Notice the Pattern

Every DFS problem has identical structure. Only the combine step changes.

```
Base Case → Ask Left Child → Ask Right Child → Combine Answers → Return Upward
```

Generic DFS Template

```
def dfs(node):
    if node is None:
        return BASE_CASE
    left = dfs(node.left)
    right = dfs(node.right)
    return COMBINE(node, left, right)
```

Everything else is just replacing `BASE_CASE` and `COMBINE`.

Complexity

	Time	Space
Balanced tree	O(N)	O(log N)
Skewed tree	O(N)	O(N)

Time: every node visited exactly once. Space: recursion stack depth.

Interview Pattern Recognition

If interviewer says: **height, depth, count, sum, maximum, minimum, search, path sum, diameter**

Immediately think: **DFS**

Then ask yourself:
1. What should `None` return?
2. What should my children tell me?
3. How do I combine their answers?

If you can answer those three questions, you've essentially solved the DFS problem.

Key Insight

Don't think: "I'm traversing the tree."

Think: "Each node asks its children for the answer, combines those answers with its own information, and returns a single result to its parent."

That mental model works for the vast majority of recursive tree problems.

Dynamic Programming Interview Cheat Sheet

Definition

Dynamic Programming (DP) solves problems by storing solutions to overlapping subproblems and reusing them.

Two Key Properties

1. **Overlapping Subproblems** — same subproblems solved repeatedly
2. **Optimal Substructure** — optimal solution can be built from optimal solutions of subproblems

Common Approaches

	Memoization (Top-Down)	Tabulation (Bottom-Up)
Direction	Recursion → Cache → Reuse	Small → Large → Final
Style	Recursive with cache	Iterative, fill table

Interview Pattern

1. **Define State** — What does `dp[i]` represent?
2. **Recurrence Relation** — Express current state using previous states
3. **Base Case** — Initialize smallest problems
4. **Transition** — Fill remaining states

Classic Problems

- Fibonacci
- Climbing Stairs
- House Robber
- Coin Change
- Knapsack
- Longest Increasing Subsequence
- Longest Common Subsequence
- Edit Distance
- Partition Equal Subset Sum
- Unique Paths

Optimization

Many DP problems reduce $O(N^2) \rightarrow O(N)$ by keeping only previous states (space optimization).

Interview Tips

Always identify **State**, **Transition**, **Base Case** before writing code.

Google ML Coding

DP appears less frequently than:
- Arrays, Hash Maps, Trees, Graphs, Sliding Window, Heap, Binary Search

A quick review of classic patterns is generally sufficient for ML-focused coding interviews.

Interview Summary

Dynamic Programming is an optimization technique that solves problems with overlapping subproblems and optimal substructure by storing intermediate results. Successful DP solutions begin by defining the state, recurrence relation, base cases, and transition before implementing either memoization or tabulation.

HashMap / Dictionary

When to Use

Keywords: `frequency`, `duplicate`, `count`, `unique`, `lookup`, `map`, `seen before`, `group by`, `anagram`

You need to:
- Count occurrences of elements
- Check if something was seen before
- O(1) lookup for existence
- Group items by a key

Classic Problems

- Two Sum (unsorted)
- Count token frequencies
- First unique character
- Anagram check / Group anagrams
- First duplicate element

Template — Frequency Count

```
from collections import defaultdict

def count_frequency(arr):
    freq = defaultdict(int)
    for item in arr:
        freq[item] += 1
    return freq
```

Template — Seen Before (Duplicate Detection)

```
seen = set()
for item in items:
    if item in seen:
        return item  # first duplicate
    seen.add(item)
```

Template — Group By

```
from collections import defaultdict

groups = defaultdict(list)
for item in items:
    groups[item.key].append(item)
```

Complexity

Operation	Average	Worst
Insert	O(1)	O(N)
Lookup	O(1)	O(N)
Delete	O(1)	O(N)

Space: O(U) where U = unique entries

Heap

When to Use

Keywords: `top k`, `largest k`, `smallest k`, `ranking`, `priority`, `kth`, `nearest`

You need to:
- Find the top/bottom K elements
- Maintain a running priority
- Efficient min/max extraction
- K is much smaller than N

Min-Heap vs Max-Heap

Need	Heap Type	Python
Top K largest	Min-heap of size K	`heapq` (keep smallest at top, evict when > K)
Top K smallest	Max-heap of size K	negate values, use `heapq`
Kth largest	Min-heap of size K	`heapq`

Classic Problems

- Top K frequent elements
- Kth largest element in array
- Merge K sorted lists
- K closest points to origin
- Top K retrieval results

Template — Top K Largest

```
import heapq

def top_k_largest(arr, k):
    min_heap = []
    for num in arr:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return min_heap  # contains K largest elements
```

Template — Top K by Score (Retrieval)

```
import heapq

heap = []
for doc_id, score in scores:
    heapq.heappush(heap, (score, doc_id))
    if len(heap) > k:
        heapq.heappop(heap)
```

Complexity

Operation	Time
Push	O(log K)
Pop	O(log K)
Top-K total	O(N log K)

vs sorting: O(N log N)

Space: O(K)

Intervals

When to Use

Keywords: `overlap`, `meetings`, `schedule`, `merge`, `insert`, `intervals`, `ranges`

Working with ranges (start, end pairs). Sorting by start time is usually the first step.

Classic Problems

- Merge intervals
- Insert interval
- Meeting rooms I / II
- Non-overlapping intervals

Overlap Rule

```
start2 <= end1  # intervals overlap
```

Merge Rule

```
[min(start1, start2), max(end1, end2)]
```

Template — Merge Intervals

```
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])  # sort by start
    merged = [intervals[0]]

    for start, end in intervals[1:]:
        last_end = merged[-1][1]
        if start <= last_end:  # overlap
            merged[-1][1] = max(last_end, end)
        else:
            merged.append([start, end])

    return merged
```

Complexity

Time: O(N log N) — dominated by sort
Space: O(N) — for output

Interview Framework

Strategy

For every problem:

```
Problem
↓
Recognize Pattern
↓
Choose Data Structure
↓
Apply Template
↓
Code
```

Step-by-Step

1. Clarify requirements.
2. State brute force.
3. Analyze complexity.
4. Identify pattern.
5. Explain optimized solution.
6. Code.
7. Test edge cases.
8. State final complexity.

Before Coding, Always Ask

1. **Pattern** — Hash? Heap? Sliding Window? DFS? Binary Search? Greedy? Two Pointers?
2. **Data Structure** — Dictionary? Set? Heap? Queue? Stack? List?
3. **Edge Cases** — Empty input? One element? Duplicates? None? Overflow? Invalid window?

Golden Rule

Don't memorize solutions. Instead ask:
- What information do I need?
- Which data structure gives it efficiently?

- What pattern naturally fits?
- What are the edge cases?
- What's the complexity?

If you can answer those five questions before writing code, you're solving the problem the way interviewers expect.

Pattern Recognition — Keyword to Pattern Map

Train your brain to recognize the pattern before writing any code.

Quick Reference Table

Keywords	Pattern
frequency, duplicate, count	**HashMap**
top k, largest k, ranking	**Heap**
substring, subarray, longest	**Sliding Window**
sorted array, threshold, first/last occurrence	**Binary Search**
overlap, meetings, schedule	**Intervals**
tree, hierarchy	**DFS / BFS**
pair in sorted array	**Two Pointers**

Pattern Recognition Drill

Read the problem, say the pattern out loud before coding:

Problem Statement	Pattern	Why
"Count how many times each word appears"	HashMap	frequency
"Find the 3rd largest element"	Heap	top k
"Longest substring with at most 2 distinct chars"	Sliding Window	substring + longest
"Find first position of 5 in sorted array"	Binary Search	sorted + first occurrence
"Merge overlapping meeting times"	Intervals	overlap + meetings
"Check if two nodes are connected in a graph"	DFS/BFS	connected + graph
"Find two numbers that sum to target in sorted array"	Two Pointers	pair + sorted

Problem Statement	Pattern	Why
"Top 5 most similar documents to query"	Cosine Similarity + Heap	semantic search + top k
"Split document into chunks of 500 tokens with 50 overlap"	Sliding Window	chunking + overlap
"Keep conversation under 4000 tokens"	Greedy Reverse	context management

Interview Discipline

1. **Read the problem twice.**
2. **Identify keywords.**
3. **Map to pattern.**
4. **Say the pattern out loud** before writing code.
5. **If no pattern matches**, consider: dynamic programming, backtracking, or greedy.
6. **If multiple patterns match**, pick the one with better complexity.

ML Coding Pattern Table

Problem Type	Pattern
Frequency Count	HashMap
Deduplication	Set
Top K Results	Heap
Retrieval	Similarity + Heap
Semantic Search	Embedding + Top K
Context Management	Sliding Window
Chunking	Sliding Window
Ranking	Sort / Heap
Recall@K	Set Operations
MRR	Ranking Scan
RAG Retrieval	Similarity + Top K + Rerank

Problem Type	Pattern
Conversation Memory	Sliding Window

ML/LLM Coding Patterns — Retrieval, Ranking & Context

Most LLM-focused coding interviews test data processing, retrieval, ranking, evaluation, context management, and vector operations — not transformer implementation.

Cosine Similarity

Keywords: `embeddings`, `similarity`, `semantic search`, `retrieval`, `nearest neighbor`

```
dot(A, B)
---------
|A| * |B|
```

```
import math

def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(y * y for y in b))
    return dot / (norm_a * norm_b)
```

Complexity: O(D) where D = embedding dimension

Dot Product

Fast retrieval scoring (when vectors are already normalized).

```
def dot_product(a, b):
    return sum(x * y for x, y in zip(a, b))
```

Complexity: O(D)

Euclidean Distance

```
import math

def euclidean_distance(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))
```

Complexity: O(D)

Top-K Retrieval

Pattern: Similarity + Heap

```
import heapq

def top_k_retrieval(query_embedding, docs, k):
    heap = []
    for doc_id, embedding in docs.items():
        score = cosine_similarity(query_embedding, embedding)
        heapq.heappush(heap, (score, doc_id))
        if len(heap) > k:
            heapq.heappop(heap)
    return sorted(heap, reverse=True)
```

Complexity: O(ND + N log K) — similarity computation + heap maintenance

---

Chunking

Keywords: `context window`, `token limit`, `split documents`

Fixed Chunk Size

```
def chunk_text(words, chunk_size):
    chunks = []
    for i in range(0, len(words), chunk_size):
        chunks.append(words[i:i + chunk_size])
    return chunks
```

Sliding Window Chunking (with overlap)

```
start = 0
while start < len(tokens):
    end = start + chunk_size
    chunk = tokens[start:end]
    chunks.append(chunk)
    start += chunk_size - overlap
```

Edge case: `overlap >= chunk_size` → infinite loop

Complexity: O(N)

Context Window Management

Problem: Keep newest messages under token budget.

Pattern: Greedy reverse traversal

```
total_tokens = 0
selected = []

for msg in reversed(messages):
    if total_tokens + msg.tokens > token_limit:
        break
    selected.append(msg)
    total_tokens += msg.tokens

selected.reverse()
```

Complexity: O(N)

Deduplication

Exact Duplicate

```
seen = set()
result = []
for chunk in chunks:
    if chunk not in seen:
        seen.add(chunk)
        result.append(chunk)
```

Near-Duplicate Detection

```
if cosine_similarity(emb1, emb2) > 0.95:
    # treat as duplicate
```

Complexity: O(N) for exact, $O(N^2D)$ for near-duplicate pairwise

---

Retrieval Metrics

Precision

```
TP / (TP + FP)
```

Recall

```
TP / (TP + FN)
```

F1

```
2PR / (P + R)
```

Recall@K

```
Relevant Retrieved in top K / Total Relevant
```

```
def recall_at_k(ranked, relevant, k):
    top_k = set(ranked[:k])
    return len(top_k & relevant) / len(relevant)
```

MRR (Mean Reciprocal Rank)

```
1 / rank of first relevant document
```

```
def reciprocal_rank(ranked_docs, relevant_docs):
    for rank, doc in enumerate(ranked_docs, start=1):
        if doc in relevant_docs:
            return 1 / rank
    return 0
```

NDCG

Measures ranking quality. Rewards relevant docs appearing early. Understand concept — usually not derived from memory in interviews.

Reranking

Pattern: Score → Sort → Return Top K

```
scores = []
for doc in docs:
    score = cosine_similarity(query_embedding, doc.embedding)
    scores.append((score, doc))
scores.sort(reverse=True)
```

Pattern Flow Summary

Problem	Pattern
Semantic search	Embedding → Similarity → Top K (Heap)
RAG retrieval	Query → Embed → Similarity → Top K → Rerank → Context
Context management	Reverse traversal + token budget
Chunking	Sliding window with overlap
Deduplication	Set (exact) / Similarity threshold (near-dup)
Ranking evaluation	Recall@K, MRR, NDCG

Last-Minute Checklist

Can I implement:
- [x] Cosine Similarity
- [x] Dot Product
- [x] Top K Retrieval (with heap)
- [x] Chunking (fixed + sliding window)
- [x] Context Truncation
- [x] Precision / Recall / F1
- [x] Recall@K
- [x] MRR

If yes, you can handle a large fraction of LLM-focused ML coding interviews.

Sliding Window

When to Use

Keywords: `substring`, `subarray`, `longest`, `shortest`, `consecutive`, `contiguous`, `window`

You need to find a contiguous subarray/substring, or optimize over a range.

Fixed vs Variable Window

Type	When	Example
Fixed window	Window size is given	"Max sum subarray of size K"
Variable window	Window size is unknown	"Longest substring without repeating chars"

Classic Problems

- Longest substring without repeating characters
- Maximum sum subarray of size K
- Minimum window substring
- Longest substring with at most K distinct characters
- Chunking with overlap

Template — Fixed Window

```
window_sum = sum(nums[:k])

for right in range(k, len(nums)):
    window_sum -= nums[right - k]     # remove leftmost
    window_sum += nums[right]         # add new right
    update_answer()
```

Template — Variable Window

```
def sliding_window(s):
    left = 0
    result = 0
    window_state = {}  # or set, counter, etc.

    for right in range(len(s)):
        # expand: add s[right] to window
        window_state[s[right]] = window_state.get(s[right], 0) + 1

        # contract: shrink from left while condition violated
        while condition_violated(window_state):
            window_state[s[left]] -= 1
            if window_state[s[left]] == 0:
                del window_state[s[left]]
            left += 1

        # update result
        result = max(result, right - left + 1)

    return result
```

Chunking with Overlap (LLM-specific)

```
start = 0
while start < len(tokens):
    end = start + chunk_size
    chunk = tokens[start:end]
    chunks.append(chunk)
    start += chunk_size - overlap  # step = chunk_size - overlap
```

Edge case: `overlap >= chunk_size` → infinite loop

Key Insight

Same template. Only the validity condition changes.

Complexity

Time: O(N) — each element added once, removed once
Space: O(K) where K is the window state size

Two Pointers

When to Use

Keywords: `pair in sorted array`, `two sum`, `sorted`, `palindrome`, `reverse`, `left right`, `meet in middle`

Array is sorted (or can be sorted), and you need to find a pair/triplet satisfying a condition, or compare elements from both ends.

Variants

Variant	Description
Opposite direction	One pointer at start, one at end, move toward center
Same direction	Fast and slow pointers (e.g., cycle detection)
Two arrays	One pointer per array, merge-style

Classic Problems

- Two sum (sorted array)
- Three sum
- Container with most water
- Valid palindrome
- Remove duplicates from sorted array

Template — Opposite Direction (Two Sum Sorted)

```
def two_sum_sorted(arr, target):
    left, right = 0, len(arr) - 1

    while left < right:
        current = arr[left] + arr[right]
        if current == target:
            return [left, right]
        elif current < target:
            left += 1
        else:
            right -= 1

    return []  # not found
```

Logic: If sum < target, increase left. If sum > target, decrease right.

Complexity

Time: O(N)
Space: O(1)